

Deployment Readiness Guide

Deployment Readiness Guide

1. What to learn before your first deploy
2. Pre-deployment checklist
3. Post-deployment verification
4. Where to go for the actual commands

Deployment Readiness Guide

This is the "before you touch the server" companion to the step-by-step [VPS Deployment \(DigitalOcean\)](#) walkthrough in the README and the `infra/scripts/deploy.sh` script. The README tells you *what commands to run*. This file tells you *what you should understand before running them*, and gives you a checklist to run through immediately before and after every production deploy.

Don't duplicate the README's command-by-command steps here — if a step changes, update it there, and keep this file about judgment, not commands.

1. What to learn before your first deploy

You don't need to be a systems administrator, but you should be comfortable enough with each of these that you're not learning it live on a production incident. Rough priority order:

Linux basics + SSH

- Navigating with a shell, editing files with `nano` / `vim`, `chmod` / `chown`, reading `df -h` / `free -h`
- SSH key auth (why we don't SSH in as `root` for daily work — see the `deploy` user in the README)
- What `sudo` does and why systemd services run as an unprivileged `deploy` user, not root

systemd (this is how the API and worker actually stay running)

- `systemctl start/stop/restart/status/enable` — this repo ships `chatbot-api.service` and `chatbot-worker.service` in `infra/systemd/`
- `journalctl -u chatbot-api -f` (tail live logs) and `journalctl -u chatbot-api --since "1 hour ago"` (look back after something broke) — this is your primary debugging tool in production, since there's no dashboard collecting these logs yet

- Why `Restart=always` + `RestartSec=5` matters (auto-recovery from a crash) and why that can also *hide* a crash-loop if you don't check `systemctl status` after a deploy

Nginx as a reverse proxy

- The difference between the three server blocks in `infra/nginx/chatbot.conf` (API domain, admin app domain, marketing/demo domain) and why `proxy_buffering off` matters specifically for this project (it breaks streaming chat responses if left on)
- `nginx -t` before every reload — a bad config blocks the reload, not just this site
- Why the widget's `/static/` path is served directly by Nginx (`alias`) instead of proxied to FastAPI — it's static file serving, no need to hit Python for it

MySQL basics

- Creating a database/user and granting privileges (`Part 2.4` of the README)
- `mysqldump` for backups and how you'd restore from one — **you should test a restore once before you need it for real**
- The difference between an Alembic migration failing on an empty dev DB vs. failing against a production DB with real rows — always assume production migrations can fail partway and know how you'd roll back (`alembic downgrade -1`)

Environment variables & secrets

- Every value in `.env.example` — know what each one does before you set it in production, not after something breaks because of one you didn't understand
- Why `JWT_SECRET` / `SESSION_SECRET` must be freshly generated per environment (never copy the dev values to prod) and what breaks if you rotate them while users have active sessions (everyone gets logged out — that's expected, not a bug)
- `ANTHROPIC_API_KEY` billing implications — this key is metered per-token; know where to check usage/spend on the Anthropic console before you go live, not after the first invoice surprises you

DNS + TLS (Cloudflare, per the README's setup)

- A records vs. proxying ("orange cloud") — proxied traffic hits Cloudflare first, which is why Nginx only needs to listen on port 80 and SSL termination happens at Cloudflare
- Why SSL mode must be **Full (strict)**, not "Flexible" — Flexible would let traffic between Cloudflare and your Droplet travel unencrypted

CORS and domain allowlisting (specific to this project)

- `ALLOWED_CORS_ORIGINS` in `.env` controls which admin/demo origins can call the API directly
- Separately, each **bot** has its own domain allowlist (see `domains` table) — that's what stops someone else embedding a client's widget on an unauthorized site. These are two different mechanisms; don't confuse "CORS is open" with "the widget is safe to embed anywhere"

Basic firewall (ufw)

- Why only SSH and Nginx ports are opened (`ufw allow OpenSSH` , `ufw allow 'Nginx Full'`) and MySQL is left unreachable from outside — the API talks to MySQL over `127.0.0.1` , nothing external

ever needs port 3306 open

This repo's release mechanics

- Read `infra/scripts/deploy.sh` end to end once — know what `--frontend-only` , `--backend-only` , and `--migrate` each skip, so you pick the right one instead of always doing a full deploy
 - Alembic migration ordering (`down_revision` chain) — if two people write migrations in parallel and both branch off the same revision, `alembic upgrade head` will fail until one is rebased
-

2. Pre-deployment checklist

Run through this before *any* production deploy, not just the first one.

Secrets & config

- ☐ `JWT_SECRET` / `SESSION_SECRET` are freshly generated (`python3 -c "import secrets; print(secrets.token_hex(32))"`), not copied from `.env.example` or dev
- ☐ `MYSQL_PASSWORD` is a strong, unique value — not the dev password
- ☐ `ANTHROPIC_API_KEY` is set, and you've checked the Anthropic console has a spend/rate limit you're comfortable with
- ☐ `ANTHROPIC_MODEL_MAIN` / `_ADVANCED` / `_FAST` are current, non-deprecated model IDs (check against Anthropic's model list — these drift over time and a stale ID will fail at request time, not at startup)
- ☐ `ALLOWED_CORS_ORIGINS` lists only the real prod domains (`https://app...` , `https://...`), no `localhost` entries left in
- ☐ `SUPERADMIN_EMAILS` is set to real admin email(s), not the `admin@demo.com` placeholder
- ☐ `APP_ENV=production` , `APP_URL` / `ADMIN_WEB_URL` point at real domains, not `localhost`

Data safety

- ☐ A recent MySQL backup exists (`mysqldump`) and you know the restore command
- ☐ Pending Alembic migrations have been run against a staging/copy of the DB first, not straight to prod
- ☐ You know the `alembic downgrade -1` path for the migration you're about to ship, in case it needs reverting

Application-level checks

- ☐ `pytest` passes locally (`services/api/tests/`)
- ☐ Any new env var introduced this cycle is added to `.env.example` (per `CLAUDE.md` 's rule — don't ship an undocumented required var)
- ☐ Rate limits (`RATE_LIMIT_PER_MINUTE`) and upload limits (`MAX_UPLOAD_MB`) still make sense for current usage patterns
- ☐ Widget `data-bot` public key flow still validates the domain allowlist (don't accidentally loosen this while debugging CORS)

Infra

- ☐ `ufw status` only allows OpenSSH + Nginx
- ☐ Cloudflare SSL mode is **Full (strict)**, DNS records for `@` , `app` , `api` all point at the current Droplet IP
- ☐ `nginx -t` passes before every reload

3. Post-deployment verification

Do this immediately after every deploy — don't assume "the script exited 0" means it's actually working.

- ☐ `curl https://api.<domain>/docs` returns 200
 - ☐ `sudo systemctl status chatbot-api chatbot-worker` — both `active (running)`, not restarting in a loop
 - ☐ `sudo journalctl -u chatbot-api --since "5 minutes ago"` — clean startup, DB connection confirmed, no tracebacks
 - ☐ `alembic current` on the server matches the latest revision in `alembic/versions/`
 - ☐ Log in to the admin dashboard, create/open a bot, send a test chat message, confirm the response **streams** (not one big delayed blob — if it's not streaming, check `proxy_buffering off` in Nginx first)
 - ☐ Paste the widget embed script on a real test page and confirm it loads and responds
 - ☐ Check one platform-level org page (`/platform/orgs`) loads correctly if you're the superadmin
-

4. Where to go for the actual commands

- Full first-time server setup: `README.md` → [VPS Deployment](#)
- Repeat/incremental deploys: `./infra/scripts/deploy.sh` (see its `--help` for flags)
- Nginx config source of truth: `infra/nginx/chatbot.conf`
- systemd unit source of truth: `infra/systemd/chatbot-api.service`, `infra/systemd/chatbot-worker.service`
- All environment variables: `.env.example` (root of `chatbot-platform/`)